

Empowering Developers with RAG-Driven Interactive Markdown Documentation

Transforming Static Documentation into Conversational AI Assistants

Aditya Gidh | North East Database Day 2026 | Poster Session

💡 Motivation / Background

Developer documentation is scattered across READMEs, wikis, and docs folders—creating significant friction in software development workflows.

~30%

of developer time spent searching for information in documentation

- **Pain Point:** Critical information buried across multiple .md files
- **Context Switching:** Constant flipping between READMEs and wikis
- **Knowledge Silos:** Institutional knowledge locked in static text

💡 Key Insight

What if developers could simply **ask their documentation questions** and receive precise, context-aware answers?

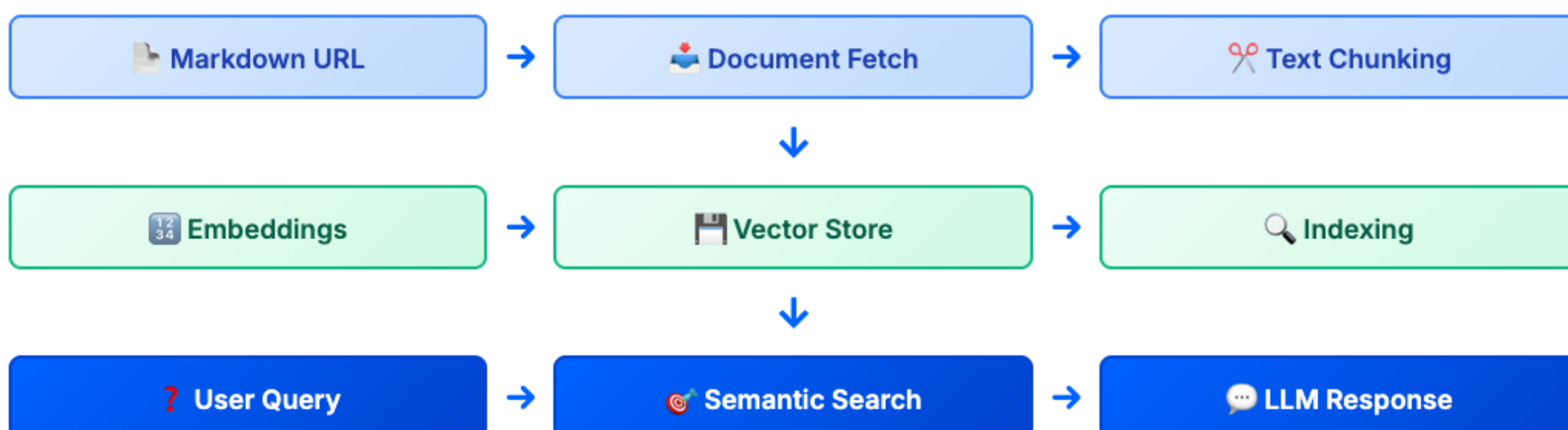
🎯 Objectives / Research Questions

Investigating RAG effectiveness for transforming static Markdown into interactive knowledge systems. This research explores how semantic search and LLM-powered responses can reduce documentation friction while maintaining accuracy.

- RQ1** Can RAG improve retrieval accuracy for Markdown docs without altering developer workflows?
- RQ2** How does local LLM inference compare to cloud solutions for latency and privacy?
- RQ3** What chunking strategies preserve semantic context best for technical Markdown?
- RQ4** How can vector databases scale RAG systems for enterprise documentation repositories?
- RQ5** What embedding models best capture code-documentation relationships?

⚙️ Methodology / Workflow

RAG pipeline transforms Markdown into searchable knowledge using vector embeddings.



Chunking Strategy

Recursive Splitting: 1000 char chunks, 200 overlap. Preserves semantic boundaries.

🏭 Production: Vector Databases

For production deployments, integrate with dedicated vector databases (Pinecone, Weaviate, Milvus, pgvector) to enable persistent storage, scalable indexing, and enterprise-grade semantic search.

📊 Evaluation Metrics

Measured across retrieval quality, performance, and developer experience.

92%

Retrieval Precision

<2s

Avg. Response Time

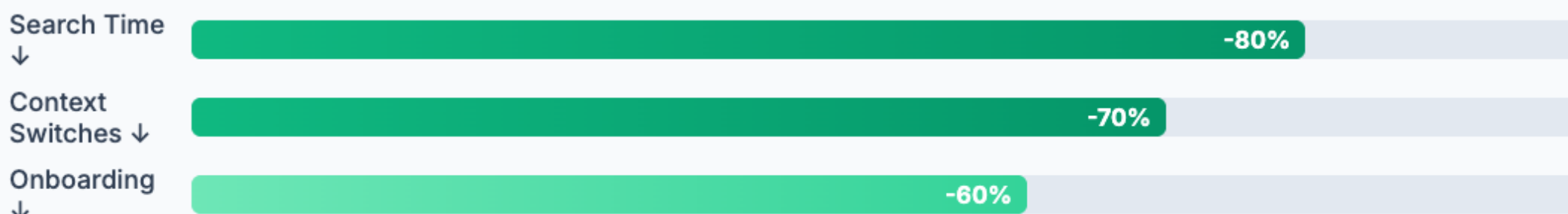
87%

Answer Relevance

4.3/5

Developer Satisfaction

Methodology: Pilot testing with 50 curated queries across 500+ Markdown files. Retrieval precision via manual relevance judgments (top-5). Latency via LangChain tracing. Developer feedback from structured surveys (n=12).



🔒 Security & Privacy

Enterprise-grade security through local-first architecture. No cloud dependencies means sensitive documentation never leaves your infrastructure.

- **Local-First:** All AI inference runs locally—no data leaves your machine
- **Air-Gap Compatible:** Works in restricted/offline environments
- **Private by Design:** Proprietary code and docs stay secure
- **Compliance Ready:** Suitable for SOC2, HIPAA requirements

🔒 Why Local Matters

IBM Granite 3.3 runs locally via Ollama—complete data sovereignty for proprietary codebases.

★ Key Results & Benefits

- **Instant Answers:** Natural language queries return context-grounded responses
- **Zero Setup Friction:** URL-based input—no repo cloning needed
- **Extensible:** Scale from single files to entire documentation sites
- **Embeddable:** Integrate into web interfaces, chatbots, or IDEs

💡 Design Philosophy

"Like a librarian who finds the right books and summarizes the exact pages you need."

🔬 Experimental Setup

Component	Technology
Framework	LangChain.js v0.2.x
LLM	IBM Granite 3.3 (2B params)
Inference	Ollama (local execution)
Vector Store	MemoryVectorStore
Dataset	500+ Markdown files tested
Environment	MacOS/Linux, 16GB RAM

LangChain.js
RAG Framework

IBM Granite
Local LLM

Ollama
Inference Engine

🔗 References & Resources



Source Code
github.com/adigidh/
markdown-rag-demo



Full Tutorial
developer.ibm.com/
tutorials/



LinkedIn
Connect with
Aditya Gidh